MODULE 1

Foundations: AI Systems & Risk Landscape

Vlad Skorokhod

© 2026 Vlad Skorokhod

SECTION 1

AI Opportunity and Risk

Where AI is already in insurance, and what's at stake

Why AI security ≠ traditional software security

The full AI attack surface: beyond the model

CIA Triad applied to AI systems

Regulatory context

AI in Insurance

Each of these six domains where AI is actively in production can be an attack target

Fraud Detection

ML models scoring claims and transactions for anomalies in real time

Underwriting Automation

Risk scoring models that inform premium pricing and policy decisions

Claims Triage

Routing and prioritizing claims based on complexity, urgency, fraud risk

Loss Prediction

Actuarial ML models estimating aggregate loss across portfolios

Customer Chatbots

LLM-powered agents handling policy queries, FNOL, renewals

Document Processing

OCR + classification for medical records, repair estimates, invoices

© 2026 Vlad Skorokhod

The Stakes Are Real

$308B

Annual insurance fraud losses (US)

FBI estimate: **~10% of all property-casualty premiums**

61%

Insurers using AI in underwriting

McKinsey 2023

High

EU AI Act risk classification

Insurance AI = high-risk systems

A compromised AI system is a financial, legal, and reputational event.

https://www.iii.org/fact-statistic/facts-and-statistics-insurance-fraud
https://insurancefraud.org/fraud-stats/
https://riskandinsurance.com/insurance-industry-increasingly-adopting-ai-technologies-study-shows/
https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2023-generative-ais-breakout-year

© 2026 Vlad Skorokhod

Why AI Systems Are Hard to Secure

AI systems are not a single component: they are a pipeline

Reality:

Multiple stages → each introduces new risks

Data, model, and API all exposed

Failures are often silent and hard to detect

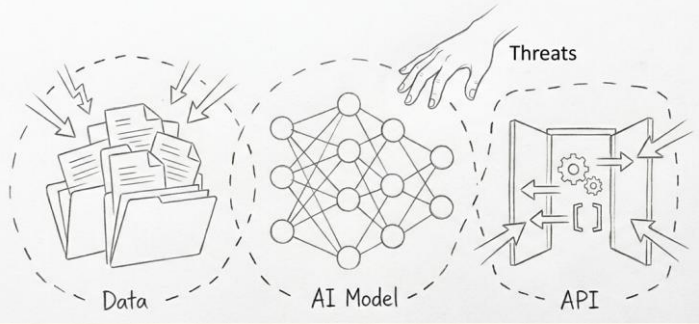
Key challenge:

Traditional security ≠ sufficient

Attacks target data + behavior, not just infrastructure

In this course:

You will attack and defend this system end-to-end



Machine Learning vs Traditional Software

Traditional Code

Instruction-based code

- Logic written explicitly
- Deterministic

```
# Rule
if claim_amount > 3000: review()
```

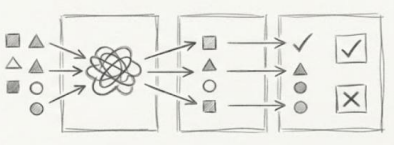
ML Model ("a function compiled from data")

ML model

- Logic learned from data
- Outputs probabilities

⚠️ **Security implication**

You **can't inspect logic**
Behavior must be **tested**



```
# Model
fraud_score = model.predict(features)
# 0.87
```

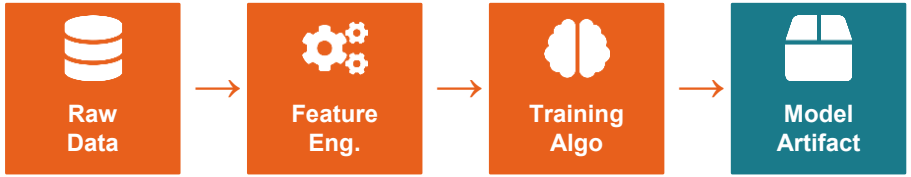
© 2026 Vlad Skorokhod

	Traditional Software	ML Model
How it's created	Hand-coded logic by developers	Learned from data via training
Decision logic	Explicit rules (if/else, formulas)	Learned weights (opaque, high-dimensional)
How it changes	Code updates + redeploy	Retrain on new data → new model artifact
Interpretability	Fully transparent	Limited (requires tools like SHAP)
Input type	Structured, schema-defined	Feature vectors (engineered or learned)
Output behavior	Deterministic	Probabilistic (scores, classes)

© 2026 Vlad Skorokhod

Training vs Inference: The Two Phases of an ML System

TRAINING PHASE · Happens offline, before deployment

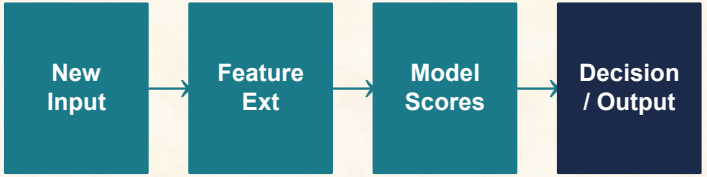


Algorithm learns from historical data: thousands of labelled examples (e.g., 'this claim = fraud', 'this claim = legitimate')

Learns to make predictions

Output: model artifact (e.g., .pkl)

INFERENCE PHASE · Live, in production



Trained model receives new data

Outputs a probability score and/or class label

Example: fraud_score=0.87

⚠️ Training → **poisoning, tampering**

⚠️ Inference → **evasion, extraction**

© 2026 Vlad Skorokhod

Machine Learning Pipeline

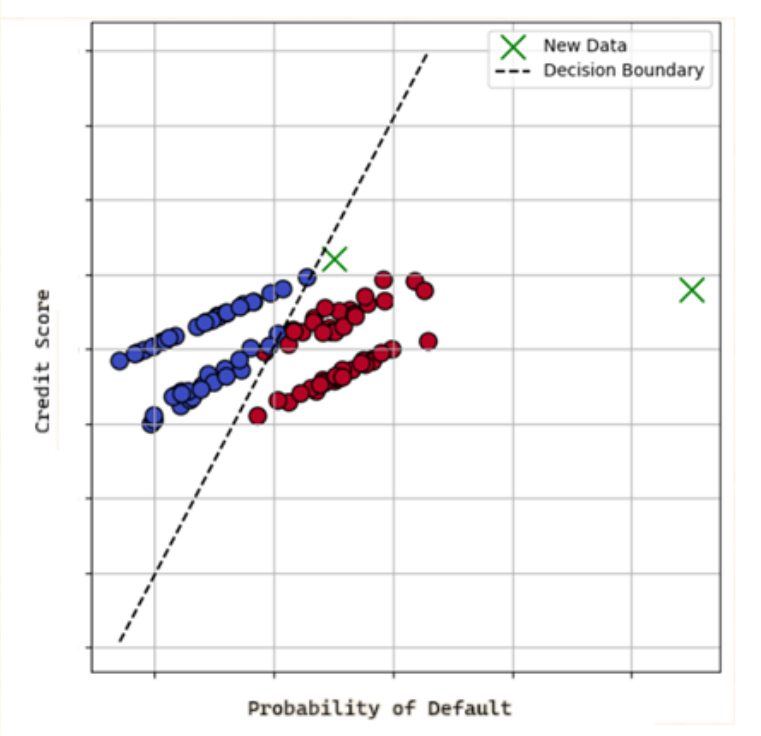
1 Data Sources	2 Feature Pipeline	3 Training Pipeline	4 Model Registry	5 Serving API	6 Monitoring & Logging
Raw inputs	ETL + feature engineering	Trains model on labelled data	Versioned storage for model artifacts.	REST endpoint	Tracks drift, query volumes etc
data poisoning	label manipulation	artifact tampering	unsigned artifact injection	adversarial inputs model extraction	log tampering

OFFLINE · Training-time attack surface

ONLINE · Inference-time attack surface

© 2026 Vlad Skorokhod

Supervised Learning Intuition



Decision Boundary (Why Attacks Work)

Predictive models don't "understand“, they separate data

- Boundary divides classes
- Close inputs → different outputs

Adversarial idea

Move input **just across boundary**

Example

Fraud score: 0.41 → flagged
Modify input → 0.39 → **approved**

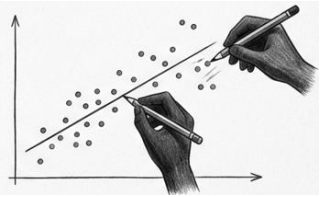
© 2026 Vlad Skorokhod

AI Systems

Classical ML

Tabular data (fraud models)

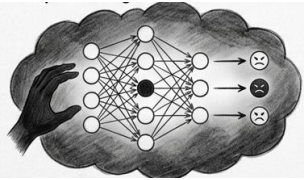
Attacks: Evasion, extraction



Deep Learning

Images, documents

Attacks: Perturbations, transfer



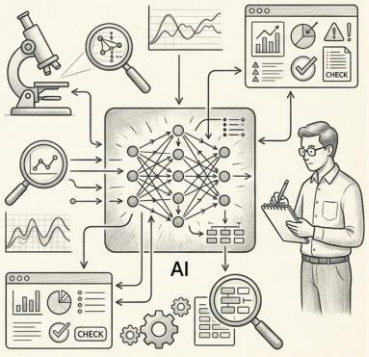
Generative AI (LLMs)

Text, chat systems

Attacks: Prompt injection, data leakage

AI Determinism and Observability

Rule-based	Predictive (Discriminative)	Generative	Agentic
Deterministic	Probabilistic training, deterministic predictions	Probabilistic (open-ended)	Adaptive + probabilistic
Fully inspectable	Inspectable features, algorithms & metrics	Partially interpretable	Lowest observability



Higher agency → higher blast radius if compromised

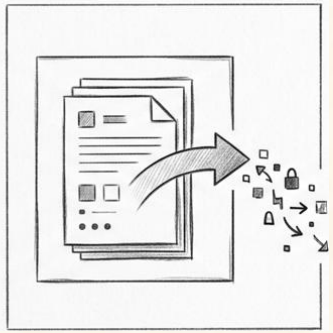
© 2026 Vlad Skorokhod

Why LLMs Are Different

- Predict **next token**, not labels
- Operate on **context window**
- Sensitive to **prompt structure**

⚠️ **New risks:**

- Prompt injection
- Data leakage via context
- Hallucination

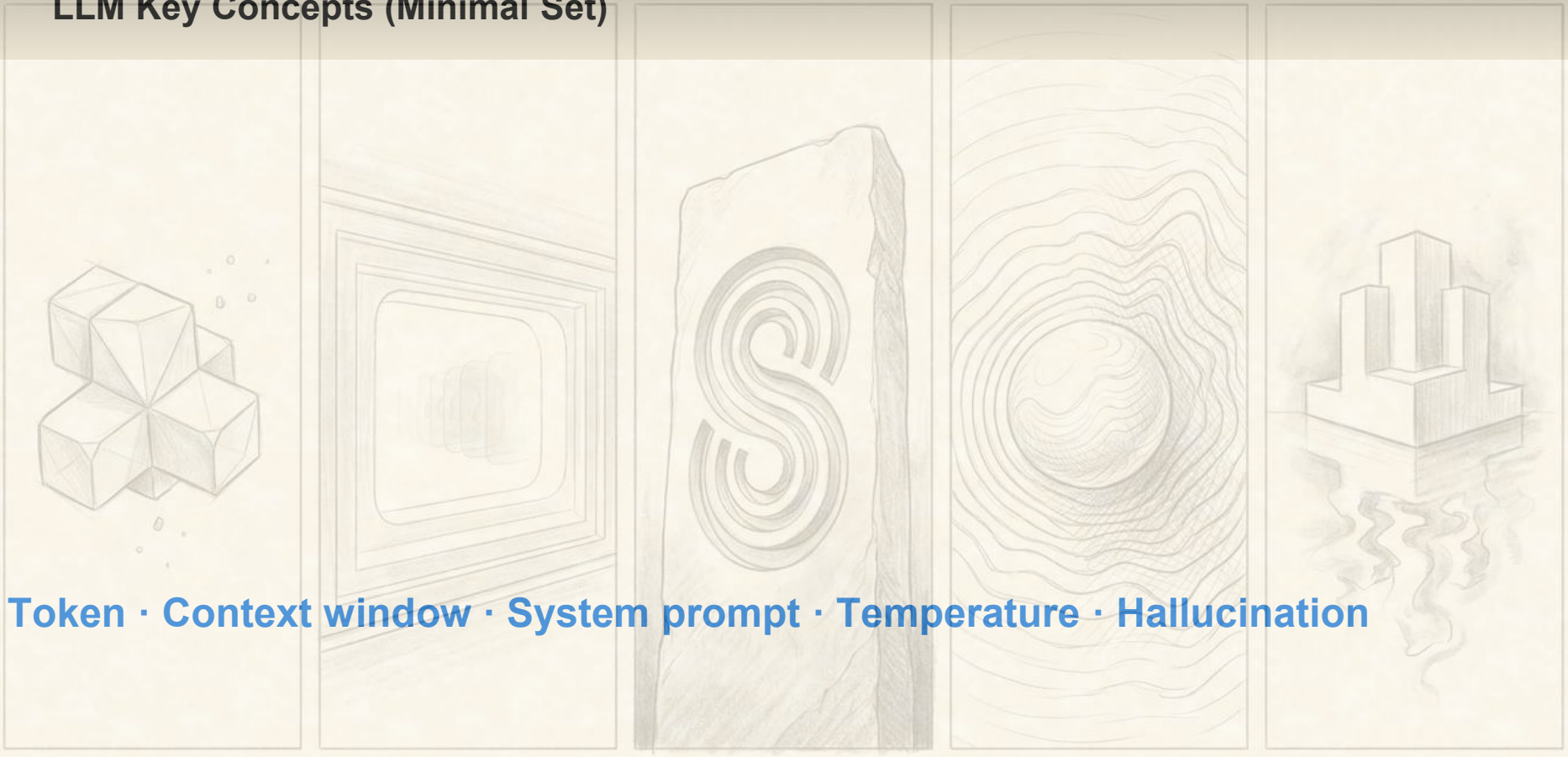


LLMs don’t differentiate between instruction and context

Everything looks like natural language

© 2026 Vlad Skorokhod

LLM Key Concepts (Minimal Set)



Token · Context window · System prompt · Temperature · Hallucination

© 2026 Vlad Skorokhod

LLMs and other generative models are probabilistic

Same input => different responses

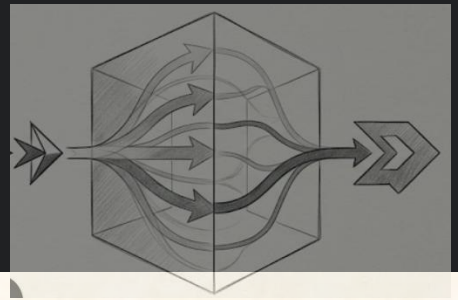
Television was invented by Philo Farnsworth in 1927, at the age of 21. He was born in Beaver, Utah in 1906. Farnsworth had an interest in electro , he designed a complete electronic system for transmitting images over a wire.

Farnsw vention and by 1927, he had created the first fully functional and completely electronic televisi fic pattern of light over a distance, which was the basis for modern television.

He app e year and began

John = 38.58%
Phil = 19.87%
a = 14.92%
the = 2.38%
\n\n = 2.10%
Scottish = 1.50%
people = 1.33%
an = 1.20%
J = 1.07%
Vladimir = 0.82%

Total: -1.62 logprob on 1 tokens
(83.76% probability covered in top 10 logits)



https://poloclub.github.io/transformer-explainer/

RAG (Most Common Enterprise Pattern)

LLM + your data (retrieved at runtime)

1. User query

↓

2. Retrieve documents

↓

3. Add to context

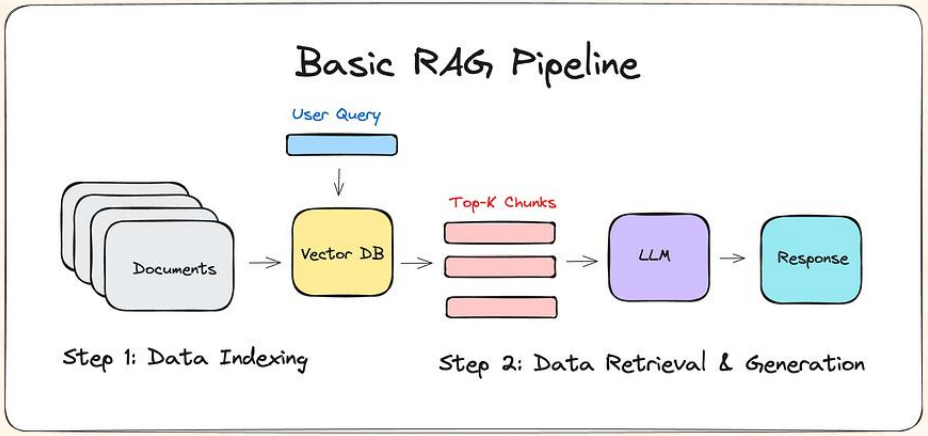
↓

4. Generate answer

⚠️ **Security risks:**

- Prompt injection via documents
- Data leakage across users

RAG (Retrieval-Augmented Generation)



Credit: https://medium.com/@drjulija/what-is-retrieval-augmented-generation-rag-938e4f6e03d1

RAG fetches relevant, factual text and appends it to model context

The Retraining Cycle (Critical for Security)

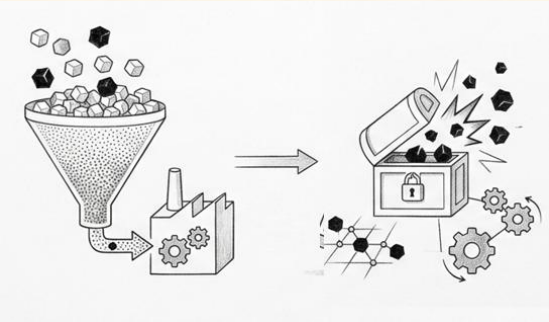
AI systems change over time

1. Collect new data
2. Retrain model
3. Validate
4. Deploy

⚠️ **Key risk**

Poisoned data → delayed impact

Attack shows up **weeks later**



Why AI Security Is Different

Traditional security

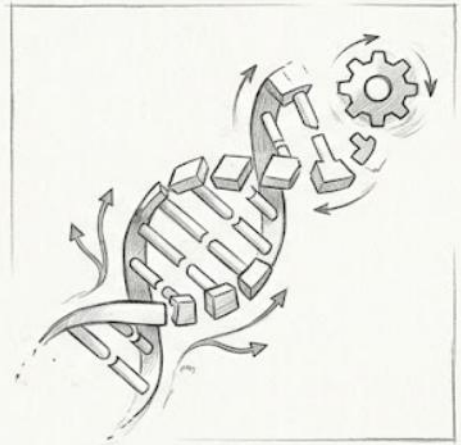
Protect code + infrastructure

AI security

Protect: Data · Models · Behavior

New properties

- Non-deterministic
- Hard to explain
- Continuously evolving



© 2026 Vlad Skorokhod

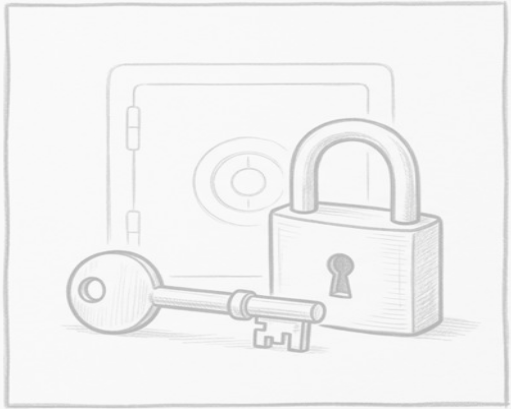
The CIA Triad in AI Systems

Confidentiality · Integrity · Availability | redefined for machine learning. A familiar framework, but AI gives each pillar a new attack surface

C Confidentiality

Prevent unauthorized access to model internals, data, or outputs

– Model theft (stolen weights/architecture)
– Model inversion (training data leakage)
– Membership inference attacks



I Integrity

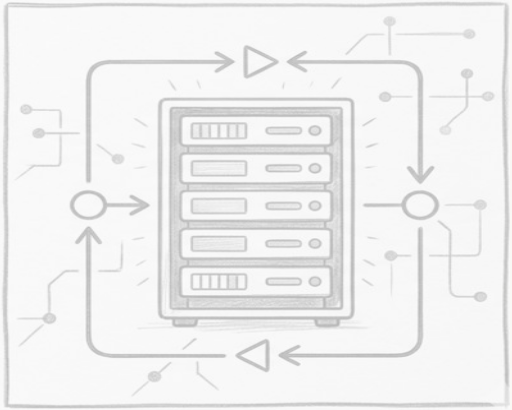
Ensure the model behaves as intended and was trained on trustworthy data

– Data/label poisoning during training
– Adversarial examples at inference time
– Backdoor/trojan model attacks

A Availability

AI system remains available and performant for legitimate users

– Model denial-of-service
– Adversarial latency exploitation
– Inference API flooding

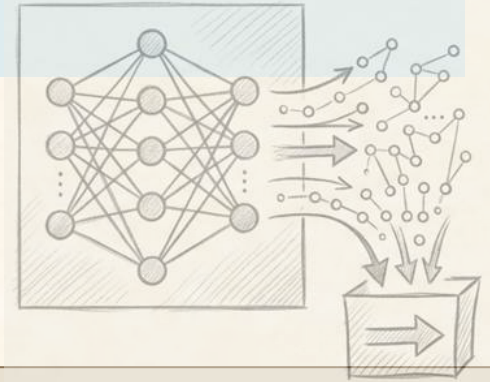


© 2026 Vlad Skorokhod

Confidentiality: What Is Unique in AI

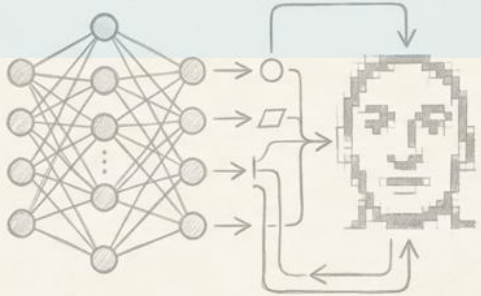
Model Theft

- Attacker probes API at scale to learn decision boundary
- Reconstructs model behavior without access to code or weights
- Impact: Loss of IP (models, data, tuning)



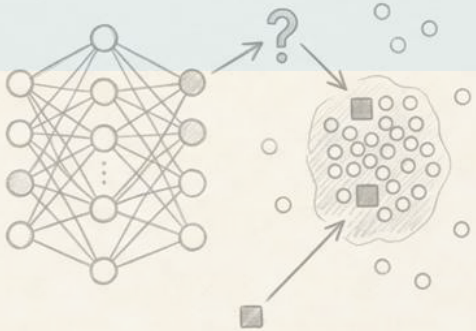
Model Inversion

- Crafted queries extract features of training data
- Can reconstruct sensitive inputs (e.g., health attributes)
- Impact: Direct data leakage (GDPR/HIPAA risk)



Membership Inference

- Attacker tests if a specific record was in training data
- Uses confidence patterns / output differences
- Impact: Privacy breach without raw data access.

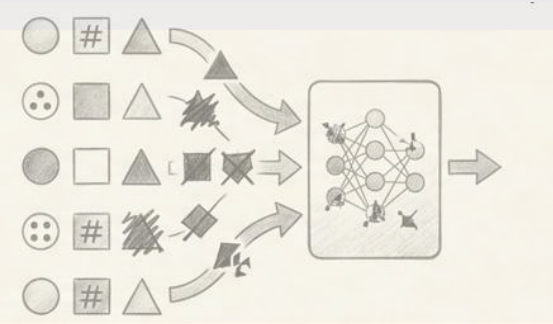


© 2026 Vlad Skorokhod

Integrity: The Training Pipeline Is an Attack Surface

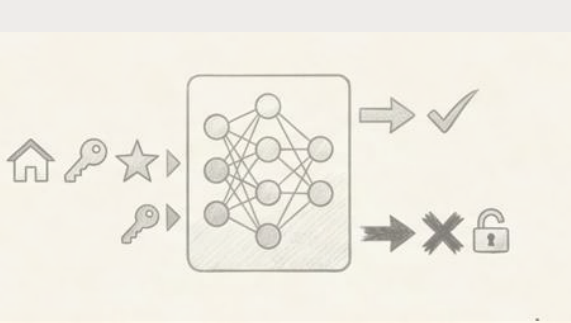
Data Poisoning

- Attackers inject crafted “clean” transactions over time to corrupt training data.
- After retraining, the model systematically trusts fraudulent behaviour.



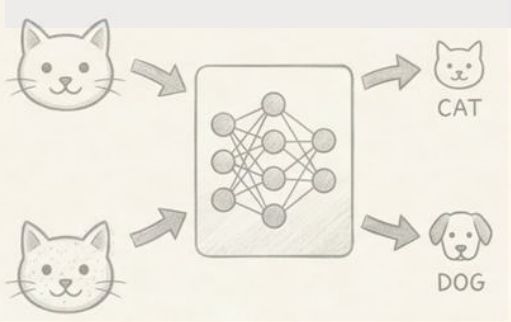
Backdoor / Trojan Attacks

- A hidden trigger is embedded during training.
- Model behaves normally but produces attacker-controlled output when triggered.



Adversarial Examples

- Slight, imperceptible input changes cause incorrect predictions at inference.
- No model access required, input alone bypasses the classifier.



© 2026 Vlad Skorokhod

Availability: AI-Specific Denial-of-Service

Sponge Attacks

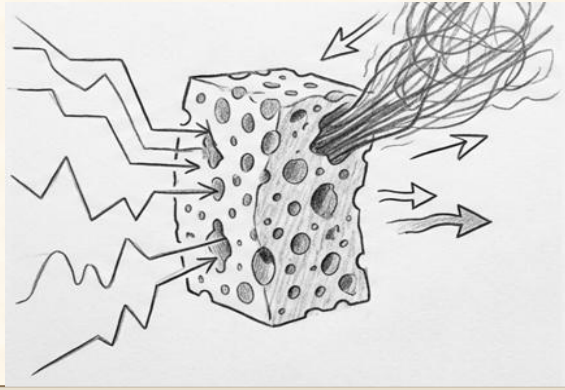
- Inputs engineered to maximize compute cost per query.
- 3ms inference → 3s latency: service degradation at minimal traffic.

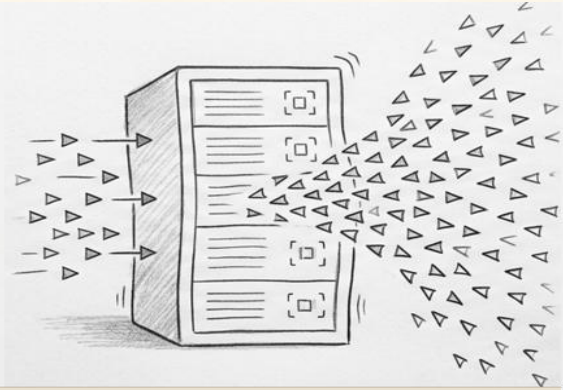
Inference API Flooding

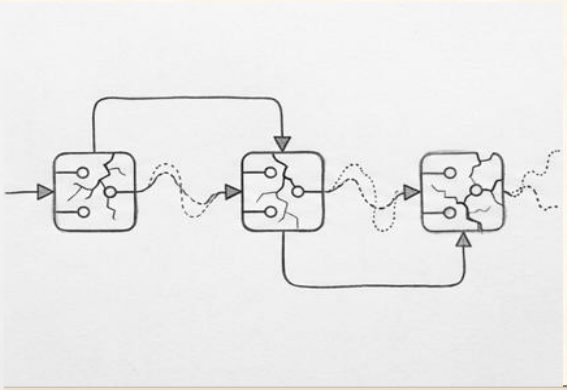
- High-rate, valid requests exhaust GPU/CPU capacity.
- No botnet required, a single client can saturate inference.

Cascading Model Failure

- Failure in one model propagates across the pipeline.
- Upstream degradation → downstream backlog, delays, and outages.







© 2026 Vlad Skorokhod

The Regulatory Landscape for AI in Insurance

Three frameworks you need to know and how they interact

US Insurance

NAIC Model Bulletin on AI Systems (2023)

Formal AI governance + executive accountability

Ongoing bias / discrimination testing required

Includes third-party and vendor models

EU / Global

EU AI Act (2024, phased enforcement)

Insurance AI = HIGH RISK classification

Pre-deployment assessment + mandatory oversight

Global scope + heavy fines (up to 7%)

Privacy

GDPR / CCPA & Training Data

Lawful basis required for training data

Data subject rights (explainability, erasure)

Data minimization + breach notification (~72h)

© 2026 Vlad Skorokhod

NAIC Model Bulletin: What It Means for Your Team

Bulletin Requirement	Engineering Implication
Written AI governance framework	Model cards, system cards, and documented risk assessments for every production model
Regular testing for bias and discrimination	Fairness metrics in CI/CD pipeline; demographic parity checks before deployment
Accountability structure for AI decisions	Audit trails, model versioning, decision logging at inference time
Vendor/third-party AI in scope	Security review of any externally-sourced models, APIs, or datasets used in production
Incident response for AI failures	AI-specific runbooks: model rollback, retraining triggers, anomaly thresholds

https://content.naic.org/sites/default/files/cmte-h-big-data-artificial-intelligence-wg-ai-model-bulletin.pdf.pdf

© 2026 Vlad Skorokhod

Key Takeaways

1. Data is an attack surface	• Treat training data as untrusted • Validate feature pipelines like APIs
2. Models can be manipulated	• Assume outputs can be adversarially influenced • Never trust LLM outputs without validation
3. Observability is critical	• Log inputs and outputs for audit and forensics
4. Models are sensitive assets	• Protect model artifacts like credentials

© 2026 Vlad Skorokhod

Quick Reference: Key AI / ML Terms

Model artifact
The saved file containing all learned weights (.pkl, .pt, .onnx, .h5). This IS the trained model.

Training data
The labelled examples used to teach the model. Quality and integrity are security-critical.

Feature engineering
Transforming raw data into numerical representations the model can process.

Inference / serving
Running a trained model on new inputs to get predictions. The production, live phase.

Retraining cycle
Periodic re-training of the model on fresh data. Creates a training-time attack window.

Hyperparameters
Config values set before training (learning rate, tree depth, etc). Not learned from data.

Decision boundary
The line/surface where the model flips its prediction. Adversarial examples live near this.

Overfitting
Model memorises training data but generalises poorly. Memorised data can leak via inversion.

Embedding
A dense vector representation of text/image. Similar things have similar embeddings.

Vector store / FAISS
Database of embeddings enabling semantic similarity search. The retrieval engine in RAG.

Prompt
The input text sent to an LLM. Includes system prompt + conversation + retrieved context.

Gradient
Directional derivative showing how input changes affect the output. Used in white-box attacks.

Model extraction
Reconstructing a model's behaviour via systematic API queries without accessing the weights.

Transfer attack
Adversarial examples crafted on a local model that also fool a different target model.

Differential privacy
Adding calibrated noise to outputs to prevent inference of training data from model behaviour.

SHAP / feature import.
Techniques for explaining which features drove a prediction. Useful for attack attribution.

Serialisation (pickle)
Python's default model save format. Executes arbitrary code on load — a security risk.

Confidence threshold
A business rule applied to the model's probability score to produce a binary decision.

© 2026 Vlad Skorokhod

SECTION 2

Case Studies

CASE 1

Data Poisoning

Fraud model

CASE 2

Model Inversion

Claims API

CASE 3

Adversarial Input

PDF documents

CASE 1

Fraud Model Poisoning

WHAT HAPPENED

Clean accounts created

Model retrains monthly on recent data

Accounts labeled “low risk”

THE ATTACK

Coordinated fraud spike

All accounts scored **≤ 0.12 risk** (threshold: 0.40)

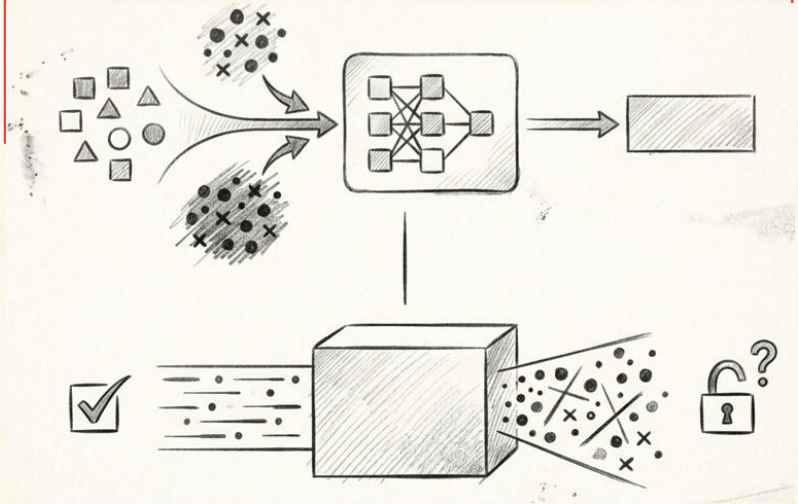
Key Insight: Model learned attacker behavior as 'normal'

IMPACT

$2.3M

fraudulent claims approved

Detected only after human audit



CASE 2

Claims Model Leakage

SCENARIO

Claims model exposed via API

Auditor granted “read-only” access

WHAT HAPPENED

180,000 queries over 3 weeks

Output: probability scores only

RESULT

Sensitive medical conditions inferred

GDPR Art. 9 violation (no breach needed)

180K

queries sent

73%

reconstruction accuracy

Art. 9

GDPR violation

Key Insight: Model outputs leak training data. No breach required

© 2026 Vlad Skorokhod

How Model Inversion Works

Simplified attacker strategy

1

Send input

Observe probability score

2

Vary one feature

One dimension at a time

3

Measure changes

Track output deltas

4

Reconstruct

Infer likely inputs

Result: Reverse-engineer sensitive features from model outputs alone

© 2026 Vlad Skorokhod

CASE 3

Adversarial PDF Example

SYSTEM

OCR + ML processes repair estimates
Claims < $2,500 → **auto-approved**

THE ATTACK

Pixel-level perturbations embedded in PDFs invisible to humans

Human sees	→	Model reads
$2,800	→	$2,200

IMPACT

31
claims approved incorrectly

$19,800
total loss

4 months
undetected

Key Insight: Model perception ≠ human perception

© 2026 Vlad Skorokhod

Adversarial Input in Document Processing

ATLAS: AML.T0043 · Component: Inference / Human Interface · CIA: Integrity

THE SCENARIO

You have built an OCR + ML to classify repair estimate PDFs.

Claims under $3,000 are auto-approved.

A body shop submits digitally modified PDFs.

Add pixel-level noise to the rendered amount fields: identical to the human eye, but shifting the ML classifier's output.

"$2,800" is rendered as $2,800 to human reviewers.

But the OCR+ML pipeline reads it as "$2,200" (below the $2,500 routing threshold that would trigger manual review).

31 claims processed over 4 months. Total overpayment: $19,800.

Discovered only when the body shop was audited for unrelated reasons.

WHAT THE ATTACK LOOKS LIKE

HUMAN SEES

REPAIR ESTIMATE

$2,800

Labor: $1,200

Parts: $1,600

→ Routed for manual review

VS

ML PIPELINE READS

REPAIR ESTIMATE

$2,200

Labor: $1,200

Parts: $1,000 ← perturbed

→ Auto-approved (< $2,500)

Pixel-level noise on rendered amount:
Invisible to human reviewers
but shifts ML classifier confidence by 34%

© 2026 Vlad Skorokhod

Debrief

Case	Attack	Layer	Impact
Poisoning	Training attack	Data pipeline	$2.3M fraud
Inversion	Privacy leakage	API / Serving	GDPR violation
Adversarial	Evasion	Input / Model	Fraud bypass

© 2026 Vlad Skorokhod

What These Three Cases Have in Common

No direct system compromise or model access required.

- All three attacks exploited a different layer: training data, the inference API, and the model's input processing.
- Perimeter security alone cannot stop AI attacks.

Typically bypass traditional controls.

- They passed network security, application security, and human review.
- Only detectable with AI-specific monitoring: data integrity checks, query rate limits, and input distribution monitoring.

© 2026 Vlad Skorokhod

What Would Have Prevented This?

Case 1 Poisoning

1. Data provenance + validation
2. Distribution anomaly detection
3. Retraining approval gate

Case 2 Inversion

1. Rate limiting + query monitoring
2. Limit output precision
3. Differential privacy

Case 3 Adversarial Input

1. Input preprocessing (compression, normalization)
2. Confidence thresholding
3. Dual validation (ML + rules)

We will discuss these defenses in detail

© 2026 Vlad Skorokhod